Silicon Club Kickoff

Advanced Digital Team

Digital Design · Physical Design · Verification · Architecture

Meet the leads



Miguel Sanchez

Architecture/PPA Lead
5th Year, EE + CS minor



Jack Payne

Digital Design Lead
4th Year, CpE



Lathan Price

Physical Design Lead
3rd year, CpE



Alejandro Paez

Verification Lead
3rd year, EE

Spring Project

Graphics Processing Unit (GPU) Core

WHAT IT IS

A small programmable core that runs one instruction across many threads at once (SIMT).

WHAT IT'S USED FOR

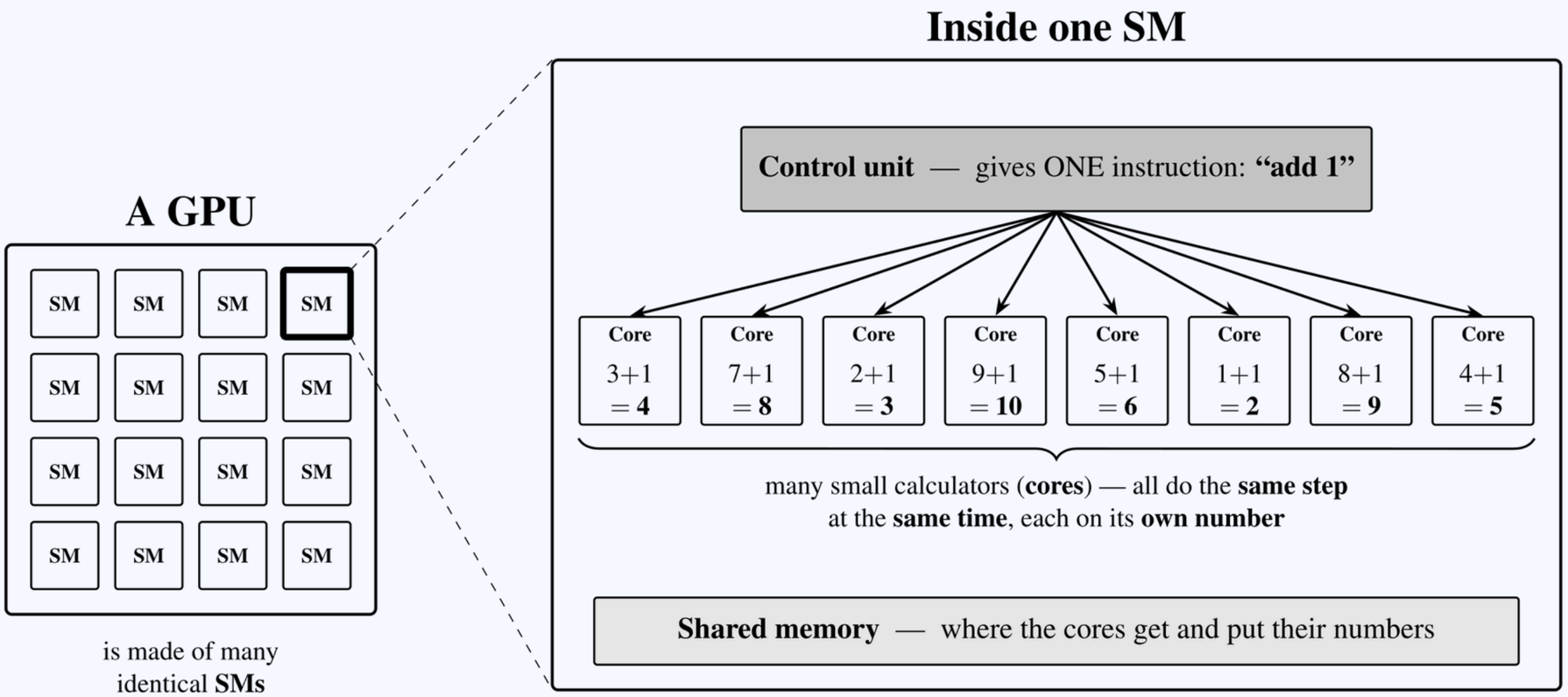
The building block of every GPU: graphics, ML and other parallel compute.

WHY IT'S A GOOD FIRST TAPEOUT

Splits cleanly into blocks (scheduler, ALU lanes, register file, memory) and scales by adding lanes.

All-digital and modular, and can scale up or down to fit. Miguel will lock down the architecture before spring.

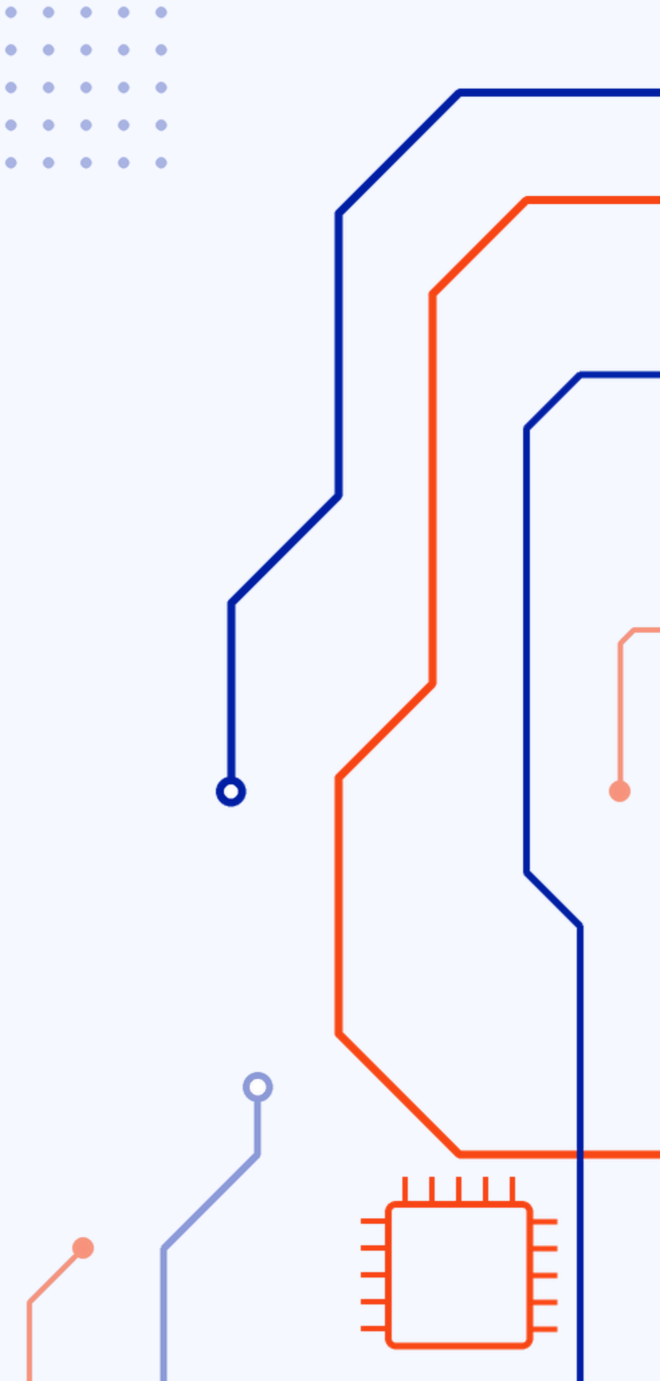
Graphics Processing Unit (GPU) Core



01

Digital Design

Presented by Jack



Digital Design: Overview

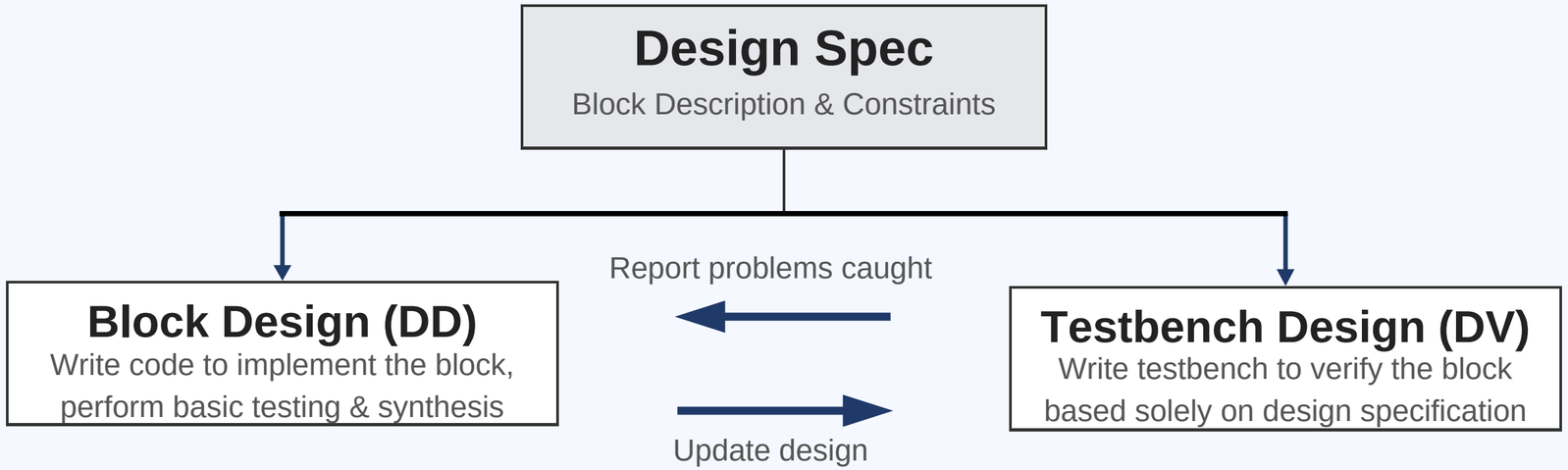
We **write the RTL that implements the logic of the design**.

- Designers **own blocks** of the GPU core
- Make implementation decisions to **meet** the PPA **constraints** of their blocks
- Hand their code to DV and **respond to problems** that they identify

Tools:

- RTL codebase will be in **SystemVerilog** and **GitHub** hosted
- IDE & personal testbench environment is **individual** (VSCode, Questa/ModelSim...)
- **Xcelium** is our shared simulation tool
- **Genus** for logical synthesis (PPA sanity checks)

Digital Design: Flow



- May have to update RTL based on PD feedback as well (timing closure infeasibility, etc)

Digital Design: Possible Blocks

01 — **SPI Host** — Used to load instruction memory & examine debug state (DFT)

02 — **Core Lane** — Heart of the GPU core, the ALU + register scratchpad instantiated multiple times

03 — **Pixel FIFO** — Accumulates the outputs of the core and conveys them to the VGA driver

04 — **VGA Driver** — Outputs pixel colors to external converter, handles VGA HSYNC/VSYNC timing

Fall Project: A Small GPU Core

For the remainder of this semester:

- Those interested in DD and DV will come together as **one RTL team** to design and verify some small elements of a GPU core
- Will help us establish a shared workflow, learn some architecture basics that we need, and generate some RTL that could be used in our final design
- Exact specification for this coming soon

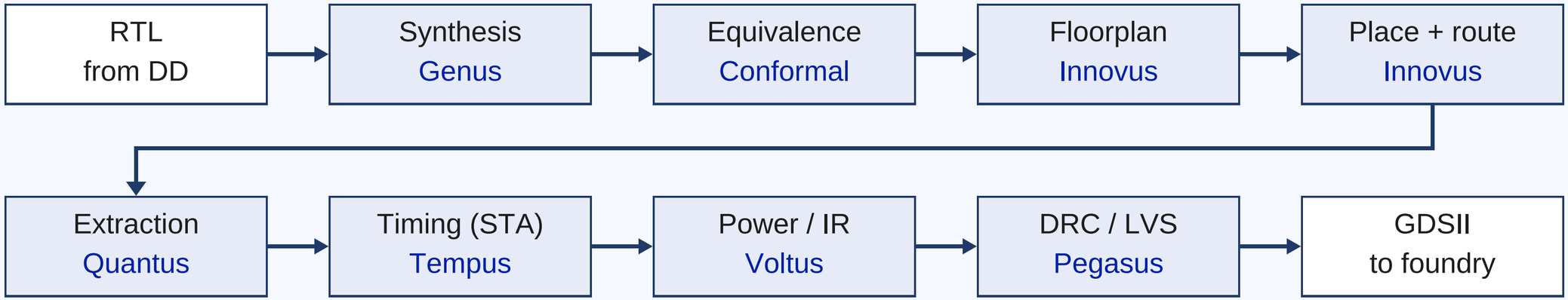
02

Physical Design

Presented by Lathan

Physical Design: Overview

PD turns the DD team's RTL into a manufacturable layout (GDSII).
Every step is scripted in .tcl/.sh.



Physical Design: Tools

Tool	What we use it for
Genus Synthesis	Maps RTL to a gate-level netlist on the standard-cell library
Conformal Equivalence (LEC)	Proves the netlist still matches the RTL after each step
Innovus Place & route	Floorplan, placement, clock tree and routing
Quantus Parasitic extraction	Extracts wire resistance and capacitance for signoff
Tempus Static timing (STA)	Checks setup and hold across corners; timing signoff
Voltus Power & IR drop	Power analysis and IR-drop checks on the power grid
Pegasus / Calibre Physical verification	DRC and LVS before the GDS goes out

Physical Design: Roadmap

01 **Training**

Complete the Cadence RTL-to-GDSII course. Required before the ALU starts.

02 **Fall: 8-bit ALU**

PD-only project: take an 8-bit ALU from synthesis through signoff on GPDK045.

03 **Spring: Tapeout**

All three tracks come together for our first tapeout.

UF doesn't offer true PD coursework, so this fall is about learning the flow together. By spring, we will be holding our own and mastering the PD flow.

Fall Project: 8-bit ALU

Opcode	Operation	Carry out
000	ADD (+ carry in)	Carry
001	SUB (− carry in)	Borrow
010	NOT	0
011	AND	0
100	OR	0
101	XOR	0
110	Shift left	Bit shifted out
111	Shift right	Bit shifted out

8-bit operands · 3-bit opcode · carry/borrow out

How it runs

1. Clone alu_8bit from the club GitHub
2. Synthesize (Genus) + LEC (Conformal)
3. Gate-level sim in Xcelium
4. Place & route in Innovus
5. Sign off: Tempus timing, DRC/LVS

WHY XCELIUM?

Simulation is normally DV's job. DV is on its own project this fall, so PD reruns the ALU testbench on the gate-level netlist to make sure synthesis didn't break anything.

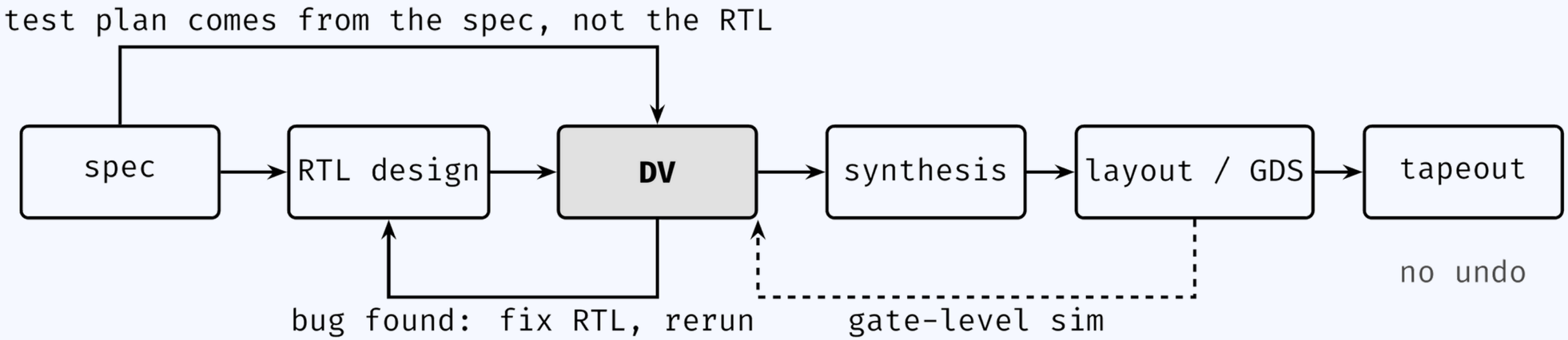
RTL → GDS on GPDK045 (no fab). Kicks off right after training and wraps before the end of the semester.

03

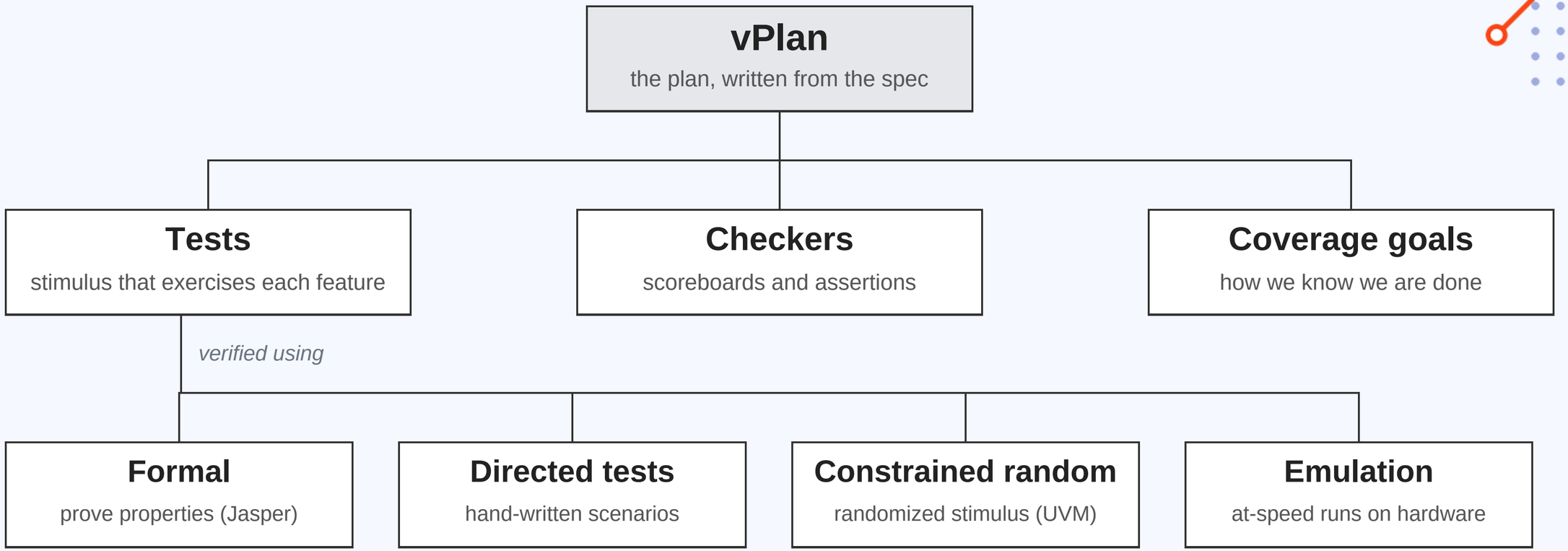
Design Verification & DFT

Presented by Alejandro

Design Verification



Design Verification: What We Work On

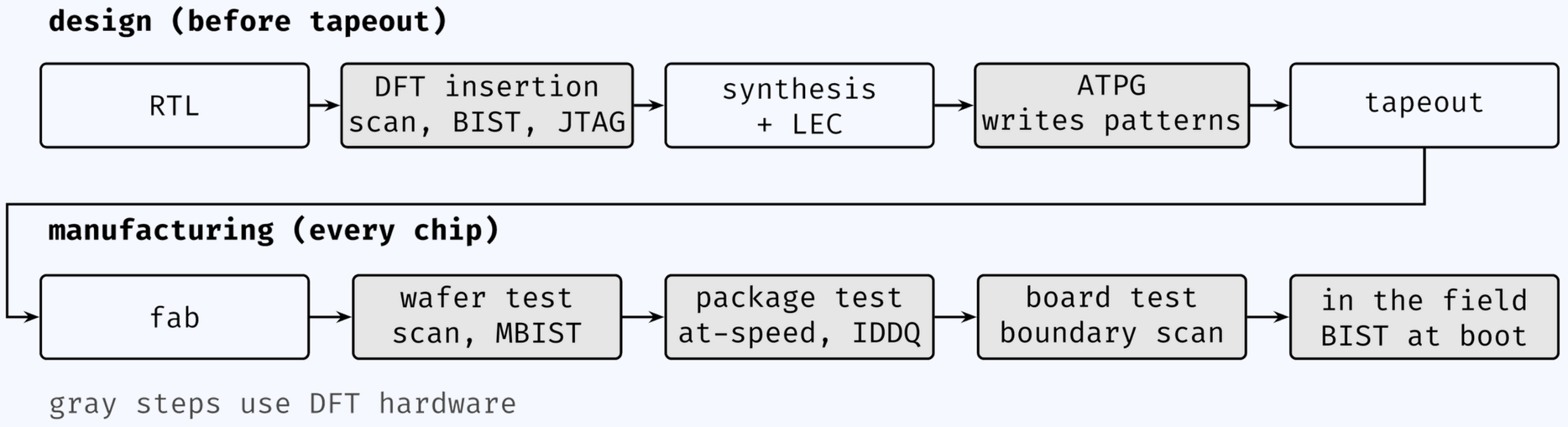


Design Verification: Levels

01	**Module**	One RTL module (FIFO, FSM, ALU). Small, fast, focused tests.
02	**Block**	Modules working together as a feature (UART, SPI). Interfaces and protocols.
03	**Chip**	Full top level: reset, boot and end-to-end flows across blocks.

Each member owns a module or block: its test plan, testbench and sign-off.

DESIGN FOR TEST



gray steps use DFT hardware

Design Verification & DFT Cadence Tools

Cadence tool	What we use it for
Xcelium Simulation	Runs our SystemVerilog/UVM testbenches, RTL and gate-level
Jasper Formal verification	Proves assertions on control logic; CDC checks
vManager Verification management	Verification plan, regressions and coverage tracking
Genus Synthesis + DFT	Synthesizes RTL and inserts scan chains for test
Modus DFT / ATPG	Generates test patterns and reports fault coverage

04

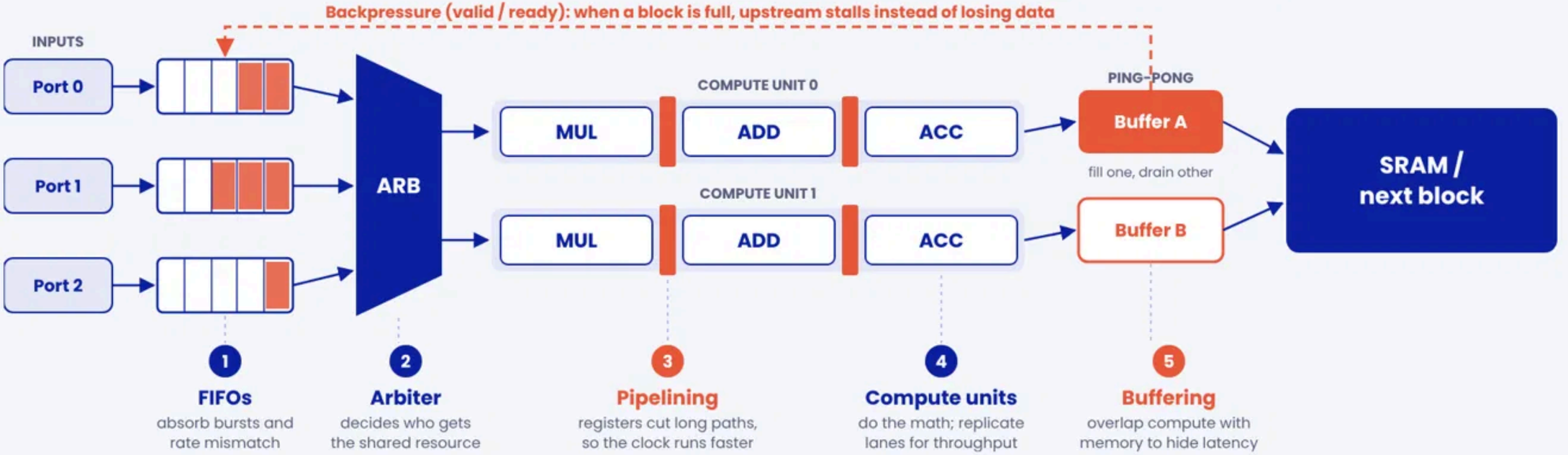
Micro Architecture and PPA

Presented by Miguel

04 · MICRO ARCHITECTURE

Building blocks of a fast design

Microarchitecture is how we **wire these blocks so data never waits**, then lay them out so **every path closes timing**.



Putting it together for max performance across DD + PD

Pipeline the critical path

- **DD** Balance logic between registers so no single stage limits the clock.
- **PD** Register long wires between blocks; use useful skew and retiming to close timing.

Decouple with FIFOs and buffers

- **DD** Every block runs at full rate; stalls and bursts don't ripple upstream.
- **PD** Place FIFO and SRAM macros along the dataflow; size them to the latency they hide, not bigger.

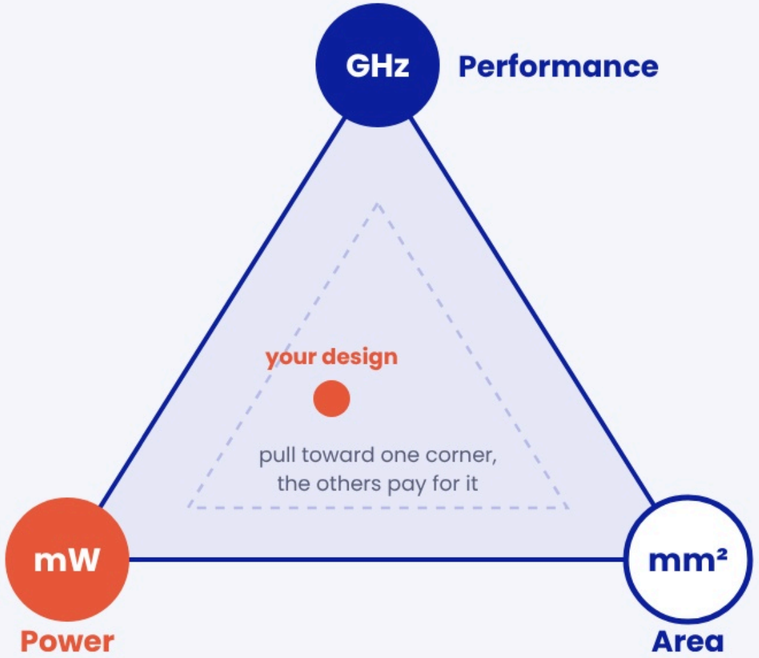
Replicate and arbitrate

- **DD** Parallel compute units multiply throughput; round-robin arbiters keep them fed fairly.
- **PD** Keep arbiter fan-in shallow and lanes side by side in the floorplan to avoid congestion.

04 · PPA

PPA: how we grade a chip

Every design decision trades **Power**, **Performance** and **Area** against each other. You can't max out all three.



GHz

Performance

How fast does it finish the work?

Measured as **clock frequency (MHz / GHz)**, latency in cycles, throughput. Limited by the slowest **critical path**, checked with static timing analysis.

mW

Power

How much energy does it burn?

Measured in **mW**. Dynamic power ≈ **α · C · V² · f** (switching) plus leakage. Drives battery life, heat and cooling.

mm²

Area

How much silicon does it take?

Measured in **mm²** of die. More area means higher cost per chip and lower yield.

Turn a knob…	Deeper pipeline	Bigger cache	Lower voltage	Clock gating
	↑ clock ↑ area, ↑ power	↑ hit rate ↑ area, ↑ power	↓ power ↓ speed	↓ power small ↑ area

04 · PPA

PPA Table

16 KB cache: how **associativity** changes timing, area and power, on ASIC (Cadence Genus) and FPGA.

ASIC synthesis

Cadence Genus SKY130 HD | ss_n40C_1v76 setup corner | 3.500 ns target | Fmax = 1 / (target - WNS)

Associativity (ways)	Target (ns)	WNS (ns)	Fmax (MHz)	Cell Area (µm²)	Total Power (W)	Violating Paths
1	3.500	-0.405	256.1	6,700,063	2.176	45,452
2	3.500	-0.242	267.2	6,447,024	2.154	43,952
4	3.500	-0.316	262.0	6,385,941	2.146	1,332
4 BEST	**3.500**	**-0.049**	**281.8**	**4,198,527**	**0.795**	**99**
8	3.500	-0.224	268.6	6,669,086	2.194	6,918
16	3.500	-0.128	275.6	6,584,580	2.275	935

FPGA implementation

FPGA Overall utilization and power

Associativity (ways)	Fmax (MHz)	LUTs (used)	LUTRAM (used)	FFs / REGs (used)	Dynamic Power (W)	Static Power (W)	Total Power (W)
1	326.6	26,349	3,448	10,826	1.618	2.975	4.592
2	301.0	30,329	3,448	11,691	1.884	2.980	4.863
4	282.3	26,331	3,514	12,609	1.514	2.973	4.486
8	269.8	26,203	3,578	13,955	1.477	2.972	4.449
16	299.9	27,679	3,580	16,466	1.475	2.972	4.447

Thanks!

Questions?

Contact us via Slack!